

MLOps - Final Project Rubric

Introduction

For your final project, you'll use your **NLP project** as the backend for a mental health support chatbot called **Serenity**.

A frontend is already provided. Your job is to build and deploy an API that powers it: classifying user intent, retrieving relevant context from a knowledge base, and generating empathetic responses using a large language model.

Details

There are two repos:

- **Frontend (provided):** <https://github.com/ishraq-hassan/chatbot-frontend> - a static chat interface deployed on GitHub Pages.
- **Backend (you build this):** You will create your own backend repo from scratch.

Steps

- **Project Planning**
 - Review and understand the frontend code (it's only 3 files).
 - Identify the API contract: what endpoints does the frontend call, and what request/response shapes does it expect?
 - Decide on your NLP approach (intent classification + RAG, fine-tuned model, prompt engineering, etc.).
 - Use the LLM provider as instructed in NLP project.
 - Use the vector database as instructed in NLP project.
 - Choose your hosting service. We recommend [Hugging Face Space](#), AWS free tier, or [Railway's free tier](#).
 - Choose your optimizations based on the service. We recommend you use ONNX-based libraries, e.g., [fastembed](#), instead of PyTorch to cut down dependency sizing.

Repository Setup (5 points)

- Create a backend/production repo for your API code.
- Use a clean, modular project structure (e.g., separate modules for models, routes, config).
- Set up a `.pre-commit-config.yaml` with at least a linter and a formatter (e.g., `ruff`, `ruff-format`).
- Include a README with setup instructions.

NLP Pipeline

- Implement an intent classification step that categorizes user messages (e.g., greeting, goodbye, mental health question, out of scope).
- Implement a response generation step. This could be:
 - RAG (retrieval-augmented generation) over a mental health dataset
 - Direct LLM prompting with a well-crafted system prompt
 - A fine-tuned model
- Handle edge cases: empty messages, out-of-scope questions, and crisis/safety scenarios.
- Explain your NLP approach and design decisions in your repo's README.

Model Serving (10 points)

- Build an API using Litestar, FastAPI, Flask, or another framework with at least:
 - POST /chat - accepts a message and returns a response.
 - POST /feedback - accepts user feedback (thumbs up/down).
 - GET /health - returns service status.
- Ensure CORS is configured to allow requests from the frontend origin.
- Add proper logging throughout your application (use Python's logging module with appropriate levels: INFO, WARNING, ERROR).
- Code should be organized and modular (not all in one file).

Unit Testing (10 points)

- Write unit tests for your API endpoints and core logic.
- Tests should cover both happy paths and error cases.
- Should be committed to your backend repo.
- MUST ALL PASS.

Containerization (10 points)

- Write a Dockerfile to containerize your application.
- Optimize your image size as much as possible.
- Use caching in the image build step. Verify that you're utilizing caching by checking the percentage of layers resulting in cache hits.
- Should be committed to your backend repo.

Monitoring Metrics (10 points)

- Instrument your API using OpenTelemetry/Axiom with at least 3 metrics:
 - 1 model/NLP-related (e.g., intent distribution, response latency, RAG retrieval scores).
 - 1 data-related (e.g., message length distribution, feedback vote ratios).

- 1 server-related (e.g., request count, error rate, uptime).
- Explain the reasoning for each metric choice in your repo's README.
- Should be committed to your backend repo.

System Monitoring (10 points)

- Set up Axiom as your observability platform.
- Use an OpenTelemetry Collector to forward events from your app to Axiom.
- Build a dashboard in Axiom that visualizes your 3 chosen metrics.
- Include a screenshot of your Axiom dashboard in your repo's README.
- Should be committed to your backend repo.

CI/CD Pipeline (10 points)

- Write a GitHub Actions workflow that runs on push to main:
 - Linting (e.g., ruff, flake8, eslint).
 - Unit tests.
 - Build and push the docker image to your chosen images registry.
 - Deployment to your hosting platform (if lint + tests pass).
- Make sure to use official & verified actions.
- Make sure to use stable versions of the used actions.
- Should be committed to your backend repo.

Deployment (10 points)

- Deploy your backend API using Hugging Face, Railway, Render, AWS, ClowdCloud, or any other suitable platform.
- The deployed API must be reachable from the frontend (CORS configured, HTTPS enabled).
- Include your deployed API URL in your repo's README.

Frontend Integration (10 points)

- Fork the provided frontend repo.
- Update the default backend URL in app.js to point to your deployed API.
- Verify that the chat interface works end-to-end (send a message, get a response, submit feedback).
- Deploy your fork using GitHub Pages.

Demo Video

- Record a screen capture showing:
 - The frontend chat working end-to-end with your backend.

- At least 2-3 different types of messages (greeting, mental health question, out-of-scope).
- The feedback buttons working.
- A brief look at your monitoring dashboard.

Deliverables

- Backend repo link
- Forked frontend repo link
- Deployed API URL
- Deployed frontend URL (GitHub Pages)
- Image layers caching verification screenshot
- Monitoring dashboard screenshot
- Demo video

Bonus Points

- Rate limiting on your API endpoints (5 points)
- Model comparison table or graph in your README (5 points)